JavaScript vs TypeScript

A practical guide to the major syntax differences, with side-by-side examples

TypeScript is a superset of JavaScript that adds static typing and a handful of extra language constructs on top of the same core syntax and runtime semantics. Every valid JavaScript file is (almost always) valid TypeScript — TypeScript mainly layers on optional type annotations, interfaces, generics, and stricter compile-time checks. The sections below walk through the major syntax differences, category by category, with equivalent JavaScript and TypeScript snippets placed side by side.

Type system JS: dynamic, runtime only TS: static, compile-time	**File extensions** JS: .js, .jsx TS: .ts, .tsx	**Compilation** JS: runs directly TS: compiles (transpiles) to JS

SECTION 01

Variable Declarations & Type Annotations

JavaScript infers types loosely at runtime and never checks them. TypeScript lets you annotate a variable's type explicitly with a colon, and will flag mismatched assignments before the code ever runs.

JAVASCRIPT

```
let age = 25;
let name = "Alice";
let isActive = true;

// no type checking - this is
// allowed and fails silently
age = "twenty-five";
```

TYPESCRIPT

```
let age: number = 25;
let name: string = "Alice";
let isActive: boolean = true;

// compile-time error:
// Type 'string' is not
// assignable to type 'number'
age = "twenty-five";
```

SECTION 02

Function Parameters & Return Types

TypeScript allows you to type both the parameters and the return value of a function, and can also mark parameters as optional or given default values with full type checking.

JAVASCRIPT

```
function add(a, b) {
return a + b;
}

function greet(name, greeting) {
greeting = greeting || "Hi";
return greeting + ", " + name;
}
```

TYPESCRIPT

```
function add(a: number, b: number): number {
return a + b;
}

function greet(
name: string,
greeting: string = "Hi"
): string {
return `${greeting}, ${name}`;
}
```

SECTION 03

Interfaces & Type Aliases

JavaScript has no built-in way to describe the shape of an object. TypeScript introduces `interface` and `type` to define reusable, checkable object shapes.

JAVASCRIPT

```
// no equivalent construct
// shape is only a convention,
// nothing enforces it
const user = {
id: 1,
name: "Alice",
email: "a@example.com"
};
```

TYPESCRIPT

```
interface User {
id: number;
name: string;
email: string;
age?: number; // optional property
}

const user: User = {
id: 1,
name: "Alice",
email: "a@example.com"
};
```

SECTION 04

Classes: Access Modifiers & Readonly

Both languages support ES6 classes, but TypeScript adds access modifiers (public, private, protected), readonly properties, and parameter property shorthand in constructors.

JAVASCRIPT

```
class Person {
constructor(name, age) {
this.name = name;
this._age = age; // 'private' is
// just a convention
}

greet() {
return "Hi, I'm " + this.name;
}
}
```

TYPESCRIPT

```
class Person {
public name: string;
private age: number;
readonly id: string = crypto.randomUUID();

constructor(name: string, age: number) {
this.name = name;
this.age = age;
}

greet(): string {
return `Hi, I'm ${this.name}`;
}
}
```

SECTION 05

Union, Intersection & Literal Types

TypeScript can describe a value as one of several types (union), a combination of types (intersection), or a fixed set of literal values — none of which JavaScript can express in syntax.

JAVASCRIPT

```
// only enforced by convention
// or manual runtime checks
function setStatus(status) {
if (["idle", "loading", "done"]
.includes(status)) {
// ...
}
}
```

TYPESCRIPT

```
type Status = "idle" | "loading" | "done";

function setStatus(status: Status) {
// status can only ever be
// one of the three literals
}

type ID = string | number; // union
type Named = { name: string };
type Aged = { age: number };
type Person = Named & Aged; // intersection
```

SECTION 06

Generics

Generics let a function, class, or interface work with multiple types while preserving type information. JavaScript has no syntax for this at all — it simply accepts anything.

JAVASCRIPT

```
function firstElement(arr) {
return arr[0];
}

const nums = [1, 2, 3];
const first = firstElement(nums);
// 'first' has no known type
// beyond generic "any"
```

TYPESCRIPT

```
function firstElement<T>(arr: T[]): T {
return arr[0];
}

const nums: number[] = [1, 2, 3];
const first = firstElement<number>(nums);
// 'first' is inferred as 'number'

interface Box<T> {
value: T;
}
```

SECTION 07

Enums

TypeScript adds an `enum` construct for a named set of constants. JavaScript has no dedicated syntax and typically simulates it with frozen objects.

JAVASCRIPT

```
// simulated with a plain object
const Direction = Object.freeze({
Up: 0,
Down: 1,
Left: 2,
Right: 3
});

let dir = Direction.Up;
```

TYPESCRIPT

```
enum Direction {
Up,
Down,
Left,
Right
}

let dir: Direction = Direction.Up;

// string enums are also supported
enum Status {
Active = "ACTIVE",
Inactive = "INACTIVE"
}
```

SECTION 08

Type Assertions & Non-null Assertions

TypeScript lets you tell the compiler to treat a value as a more specific type than it inferred, using `as`, or to assert a value is not null/undefined with `!`. Neither has a JavaScript equivalent.

JAVASCRIPT

```
// no type system, so no assertions -
// you just use the value directly
const input =
document.getElementById("email");
input.value = "test@example.com";
```

TYPESCRIPT

```
const input =
document.getElementById("email") as
HTMLInputElement;
input.value = "test@example.com";

// non-null assertion operator
function process(value?: string) {
console.log(value!.toUpperCase());
}
```

SECTION 09

Modules: Type-only Imports & Exports

Both languages share the ES module `import`/`export` syntax. TypeScript extends it with `import type` / `export type` to make clear an import exists only for type-checking and is erased at compile time.

JAVASCRIPT

```
// modules.js
export function add(a, b) {
return a + b;
}

// app.js
import { add } from "./modules.js";
```

TYPESCRIPT

```
// modules.ts
export interface Point { x: number; y:
number; }
export function add(a: number, b: number):
number {
return a + b;
}

// app.ts
import { add } from "./modules";
import type { Point } from "./modules"; //
erased at build
```

SECTION 10

Tuples & Fixed-shape Arrays

TypeScript adds tuple types — arrays with a fixed length where each position has a known, specific type. JavaScript arrays are always just arrays, with no positional typing.

JAVASCRIPT

```
// just a regular array,
// length and types are not fixed
let point = [10, 20];
point.push("oops"); // allowed
```

TYPESCRIPT

```
let point: [number, number] = [10, 20];

// compile-time error: too many
// elements / wrong type
point.push("oops");

// labeled tuple elements
type Range = [start: number, end: number];
```

SECTION 11

Decorators (Experimental Metadata Syntax)

TypeScript (and now Stage-3 JavaScript) supports decorators — special annotations that can modify classes, methods, or properties. In plain JavaScript projects this typically requires a Babel plugin.

JAVASCRIPT

```
// no native syntax; commonly done
// with a Babel plugin or by hand
class Greeter {
greet() {
console.log("Hello!");
}
}
```

TYPESCRIPT

```
function log(target: any, key: string,
descriptor: PropertyDescriptor) {
console.log(`Calling ${key}`);
return descriptor;
}

class Greeter {
@log
greet() {
console.log("Hello!");
}
}
```

Note: TypeScript's extra syntax is entirely optional and erased at compile time — the compiler (tsc) strips out all type annotations, interfaces, and similar constructs, producing plain JavaScript that runs anywhere JavaScript runs. This means every TypeScript feature above is a development-time and tooling benefit, not a runtime one.